Course: Enterprise Real-Time Data Engineering
Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)
Course Portal: fakhr.me/KafkaSpark
Code & Labs Repository: github.com/fakhruldeen/KafkaSpark

Session S12: Hands-On Lab Walkthrough & CLI Guide

Production Operations, Monitoring & Capstone Defense

1. Lab Objectives

In this final capstone engineering lab, you will validate, monitor, and defend the complete enterprise data platform:
1. Audit cluster telemetry across all 8 Docker containers (Kafka brokers, Schema Registry, Kafka Connect, Spark Master, Spark Worker).
2. Triage production JMX health metrics and verify that `UnderReplicatedPartitions == 0` and `OfflinePartitionsCount == 0`.
3. Execute Chaos Engineering drills by taking down a Kafka broker and proving zero-downtime cluster survival and automatic leader failover.
4. Measure and mitigate consumer lag on critical retail topics.
5. Execute the comprehensive Capstone Test Suite (T1 through T5) and achieve full certification readiness.

2. Environment Setup & Cluster Health Verification

Ensure that all 8 platform services are running and healthy:

```
cd "${LAB_ROOT:-delivery_materials/00_Lab}" 2>/dev/null || cd 00_Lab || cd ../../00_Lab

source .venv/bin/activate
if [ -d "${HOME}/.local/share/jvm/jdk-17.0.21+9" ]; then
  export JAVA_HOME="${HOME}/.local/share/jvm/jdk-17.0.21+9"
fi
export PATH="←!—KATEX_INLINE_0⟶{PATH}"

# Verify container statuses
docker ps --format "table {{.Names}}    {{.Status}} {{.Ports}}"
```

3. Step 1: Kafka Cluster Telemetry & Partition Health

Inspect partition distribution and leader assignments across topics:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh   --bootstrap-server kafka1:9092   --d
escribe --topic orders.events.v1
```

Confirm that:

- Every partition has an active Leader.
- `Replicas` and `Isr` match perfectly (e.g. `Replicas: 1,2,3  Isr: 1,2,3`).

---

4. Step 2: Chaos Engineering Drill (Broker Failover & Recovery)

Simulate an unexpected hardware failure on `kafka2`:

```
# 1. Stop broker 2
docker stop kafka2

# 2. Immediately describe the topic to observe leader election
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh   --bootstrap-server kafka1:9092   --d
escribe --topic orders.events.v1
```

Observe:

- `Isr` shrinks to `1,3`.
- Any partition that previously had `Leader: 2` is immediately reassigned to `1` or `3` with zero write interruptions for `acks=all` producers.

Restore broker 2:

```
docker start kafka2
sleep 5

docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh   --bootstrap-server kafka1:9092   --d
escribe --topic orders.events.v1
```

Confirm that broker 2 catches up from the leader's write-ahead log and rejoins the ISR list (`Isr: 1,3,2`).

---

5. Step 3: Consumer Group Lag Triage

Inspect consumer lag across all active consumer groups:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-consumer-groups.sh   --bootstrap-server kafka1:9
092   --describe --group retail-order-processors
```

Key metrics to evaluate:
- `LOG-END-OFFSET`: Highest committed offset in the partition.
- `CURRENT-OFFSET`: Latest offset read by the consumer.
- `LAG`: Difference between `LOG-END-OFFSET` and `CURRENT-OFFSET`. An escalating lag indicates downstream processing bottlenecks.

---

6. Step 4: Executing the Full Capstone Test Suite (T1–T5)

Run the comprehensive end-to-end integration test suite validating all 5 architectural acceptance tests:

```
pytest tests/test_capstone_pipeline.py -v
```

Expected Output:

```
tests/test_capstone_pipeline.py::test_t1_schema_registry_contract PASSED       [ 20%]
tests/test_capstone_pipeline.py::test_t2_cdc_mysql_ingestion PASSED            [ 40%]
tests/test_capstone_pipeline.py::test_t3_spark_streaming_watermarking PASSED   [ 60%]
tests/test_capstone_pipeline.py::test_t4_dlq_poison_pill_routing PASSED        [ 80%]
tests/test_capstone_pipeline.py::test_t5_broker_failover_resilience PASSED     [100%]
============================== 5 passed in 12.45s ==============================
```

---

7. Step 5: Master Verification Script

Run the session verification script:

```
bash Trainer/S12_05_verify_s12.sh
```

Expected Output:

```
[PASS] S12 Production Operations & Capstone Defense fully certified.
```

🌟 Gold Stretch Challenge: Zero-Downtime Rolling Cluster Upgrade

> **Distinguished Engineer Challenge:** If you complete the core lab before time expires, execute a simulated zero-downtime rolling restart:
> 1. Identify the current KRaft Active Controller:
> `bash`
> `docker exec -it kafka1 /opt/kafka/bin/kafka-metadata-shell.sh --snapshot /tmp/kraft-combined-logs/__cluster_metadata-0/00000000000000000000.log --print-controller`
> 2. Sequentially restart non-controller brokers first (`kafka2` , then `kafka3`), waiting after each restart until `UnderReplicatedPartitions == 0` .
> 3. Finally, restart the Active Controller broker (`kafka1`) and observe how the KRaft quorum elects a new leader controller within 250 milliseconds with zero dropped messages on active producers.